darkmpc: the two ring extensions, proved

Product checks in the 64-bit ring, and the MAC-checked shuffle

Tave La Rocque

6 November 2026

darkmpc's protection against a cheating server rests on two published techniques, each analysed by its authors in a different setting:

- **Product checks.** Furukawa, Lindell, Nof and Weinstein (FLNW; Eurocrypt 2017, ePrint 2016/944) verify multiplications with triples that are produced by cut-and-choose and checked in buckets. Their analysis is for Boolean circuits. darkmpc uses it for AND gates (exactly as published) and also for multiplications in $\mathbb{Z}_{2^{64}}$, which is the first extension.
- **Shuffle check.** Asharov, Hamada, Ikarashi, Kikuchi, Nof, Pinkas and Tsuji (CCS 2022) make the shuffle tamper-evident with MACs and a random linear check, analysed over a field. darkmpc does the MAC arithmetic in $\mathbb{Z}_{2^{128}}$, using the lifting idea of SPDZ2k (Cramer, Damgård, Escudero, Scholl and Xing, CRYPTO 2018), inside the honest-majority replicated shuffle. That combination is the second extension.

This document proves the two soundness statements those extensions need, step by step. It does not give a full simulation proof of the whole malicious protocol; Section 5 says what such a proof would combine and what is left. These are the author's proofs and have not been reviewed by an expert.

1. Setting and the two facts everything uses

Three servers P_0, P_1, P_2, indices mod 3, at most one corrupt and **malicious**: it may send anything. A value x in a ring R (here $\mathbb{Z}_{2^{64}}$, $\mathbb{Z}_{2^{128}}$ or $\mathbb{Z}_2^{64}$) is shared as $x = x_0 + x_1 + x_2$, and P_j holds (x_j, x_{j+1}).

Every summand has an honest holder. Summand x_j is held by P_j and P_{j-1}. With one corrupt server, at least one of the two is honest. So we *define* the value of a sharing by the honest holders' copies:

$$x := x_0 + x_1 + x_2, \quad \text{each } x_j \text{ taken from an honest holder.}$$

When both holders are honest they agree, because honest servers follow the protocol. A corrupt server's own copies are irrelevant to the value; they matter only through the messages it sends.

Fact 1 (checked openings). In a checked opening, P_i receives its missing summand x_{i+2} from both of its holders, P_{i+1} and P_{i+2}, and aborts if the copies differ.

one copy is honest $\implies$ an honest receiver either aborts or learns the true x.

This is the replicated-sharing reveal of Eerikson et al. (ITC 2020, Section 2.4). Every opening in malicious mode is checked.

Fact 2 (additive attack). In the multiplication protocol, P_j computes

$$z_j = x_j y_j + x_j y_{j+1} + x_{j+1} y_j + \alpha_j$$

and sends it to P_{j-1}, where $\alpha_0 + \alpha_1 + \alpha_2 = 0$. Let P_c be corrupt and suppose it sends $z_c + \delta$ instead of z_c.

The summand z_c is held by P_c and P_{c-1}, and P_{c-1} is honest, so by the definition above its value is what P_{c-1} received:

$$z_c^{\text{def}} = z_c + \delta.$$

The other two summands, z_{c+1} and z_{c+2}, are computed by honest servers from honest copies, so they are correct. Summing, and using that the correct summands add up to xy:

$$z = xy + \delta.$$

So whatever the corrupt server does, the product is a valid sharing of $xy + \delta$ for some error δ in R. The error is a function of the corrupt server's view and coins. That view is independent of the secret inputs (docs/design/leakage-audit.md, Steps 3 and 4), so δ is too. The argument uses no division, so it holds in every ring above. For $\mathbb{Z}_2^{64}$ with XOR it is FLNW's Lemma 2.4 applied to each bit.

2. Extension 1: product checks in $\mathbb{Z}_{2^{64}}$

2.1 What the protocol does (crates/mpc/src/verify.rs)

Every multiplication $z = x \cdot y$ made during the auction is recorded. Before anything is opened, the recorded products are verified in batches of up to $N \leq 2^{18}$:

1. Produce $NB + C$ triples (a, b, c), with a, b random sharings and c computed by the multiplication protocol. Here $C = B$.
2. Open a fresh random seed (checked), and expand it into a permutation of the triples.
3. Open the first C triples completely and check $c = ab$ for each.
4. Split the rest into N buckets of B. Check the first triple of each bucket against every other triple in it, using the pair check below.
5. Check each recorded product against the first triple of its own bucket, using the same pair check.

The pair check. To check (x, y, z) against (a, b, c), open $\rho = x - a$ and $\sigma = y - b$, compute locally

$$t = z - c - \sigma a - \rho b - \rho\sigma,$$

and open t. The check passes if $t = 0$.

2.2 The pair check outputs exactly the difference of the errors

Lemma 1. Let $z = xy + \delta_z$ and $c = ab + \delta_c$. If no honest server aborts, the opened t equals $\delta_z - \delta_c$.

Proof.

By Fact 1, the opened ρ and σ are the true values:

$$\rho = x - a, \qquad \sigma = y - b.$$

Every step from there to t is local and linear, so t's value is computed from the values of z, c, a, b:

$$t = z - c - \sigma a - \rho b - \rho\sigma.$$

Substitute $x = \rho + a$ and $y = \sigma + b$ into xy and expand:

$$xy = \rho\sigma + \rho b + \sigma a + ab.$$

Write z and c with their errors:

$$t = (xy + \delta_z) - (ab + \delta_c) - \sigma a - \rho b - \rho\sigma.$$

Replace xy by its expansion. Every term without a δ cancels:

$$t = \delta_z - \delta_c.$$

The opening of t is checked as well, so the honest servers see this exact value. □

No inverse was used, so this holds in any commutative ring.

Consequence.

$$\text{the pair check passes} \iff \delta_z = \delta_c.$$

A check never passes "by luck". It passes exactly when the two errors are equal, which the corrupt server either arranged or did not.

2.3 The bucket game reduces to the Boolean one

Call a triple *bad* if its error $\delta \neq 0$.

What the corrupt server needs. The triples' errors are fixed in Step 1, before the seed is opened, so they are a fixed vector $(\delta_1, \ldots, \delta_{NB+C})$ when the permutation is drawn. For a bad triple to be used in Step 5 with no abort, three things must all happen:

- **(a)** Each of the C opened triples has $\delta = 0$. Opening shows the true $(a, b, ab+\delta)$, by Fact 1.
- **(b)** In every bucket, every triple has the same error as the bucket's first triple. This is Lemma 1, applied to each check in Step 4.
- **(c)** Some bucket's first triple is bad.

Lemma 2 (the ring adds nothing). Let W_R be the event (a) ∧ (b) ∧ (c). Let W_2 be the event that the opened triples are all good, every bucket is all good or all bad, and some bucket is all bad. Then

$$W_R \subseteq W_2.$$

Proof.

Assume W_R.

By (a), no opened triple is bad.

By (b), all triples in a bucket share the first one's error. If that error is 0, the whole bucket is good. If it is nonzero, the whole bucket is bad.

By (c), at least one bucket's first triple is bad, so that bucket is all bad.

That is exactly W_2. □

The containment is usually strict, because W_2 ignores which nonzero error each bad triple carries, while W_R needs them equal within each bucket.

W_2 is exactly the event FLNW bound for Boolean triples, where every bad triple has the same error. Their bound is a statement about a uniformly random permutation and holds for every fixed set of bad triples.

FLNW Theorem 5.3. For $C \geq B$, and any fixed set of bad triples,

$$\Pr[W_2] \leq \frac{N}{\binom{NB+C}{B}}.$$

Theorem A. In one verified batch of N products, the probability that a product with $\delta_z \neq 0$ passes Step 5 without any honest server aborting is at most

$$\frac{N}{\binom{NB+C}{B}} + \varepsilon_{\text{coin}},$$

where $\varepsilon_{\text{coin}}$ bounds how far the permutation is from uniform (below).

Proof.

If W_R does not happen, then either some check in Steps 3 and 4 aborts, or every first triple has $\delta_c = 0$.

In the second case, Lemma 1 says the Step 5 check of a product passes exactly when $\delta_z = \delta_c = 0$. So every product with an error is caught.

Hence a product with an error can pass only inside W_R.

By Lemma 2, $\Pr[W_R] \leq \Pr[W_2]$, and FLNW Theorem 5.3 bounds that for a uniform permutation.

The real permutation comes from a 128-bit seed expanded with AES and Fisher-Yates, which adds $\varepsilon_{\text{coin}}$. □

The coin term. The seed is a checked opening of a random sharing. One of its summands comes from the key the corrupt server lacks, so the seed is pseudorandom to it until it is opened, and it is opened only after the triples are fixed. Replacing AES by a random function costs the PRF advantage. A distinguisher can test for W_2 with these fixed errors without knowing the key, because it only needs to evaluate the permutation its oracle produces. Given uniform words, the permutation is exactly uniform, because Fisher-Yates draws each index by rejection sampling (`uniform_below` in crates/mpc/src/perm.rs). So $\varepsilon_{\text{coin}}$ is just the PRF advantage.

A bug this proof found. The sampler used to take each index as a 64-bit word mod $(i+1)$. That is biased by up to $(i+1)/2^{64}$ per draw, and across a permutation of M items the distance from uniform is bounded only by about $M^2/2^{65}$. Its comment argued this was negligible for $M \le 2^{17}$, the batch limit, but the triple permutation has $M = NB + C$, up to about 2^{21} items. There the bound is about 2^{-23}, nowhere near the 2^{-56} target. The bias is probably far smaller in practice, but a proof can only use what it can bound. Rejection sampling removes the term entirely, at a cost of almost nothing: a draw is rejected with probability below 2^{-43} at these sizes. `crates/mpc/tests/proof_checks.rs` and the unit tests of `uniform_below` cover the new sampler.

2.4 AND gates, and words

AND gates run the same protocol on 64-bit words with XOR. FLNW's analysis treats single bits. Treating each word as one item, a word is bad if its error word is nonzero, and the pair check passes exactly when the two error words are equal (Lemma 1 in $\mathbb{Z}_2^{64}$). So Lemma 2 and Theorem A hold with N counted in **words**, not gates. The security accounting in the code subtracts 6 bits for 64 gates per word. By this argument that subtraction is not needed, so the reported levels (at least 43.9 bits) are conservative by about 6 bits. They are left as reported.

2.5 Privacy of the checks

A failed check aborts, and that abort must not depend on secrets. That is the selective-failure attack Section 3 exists to stop for shuffles. Here it cannot happen. By Lemma 1, whether a check passes depends only on the errors, and those are functions of the corrupt server's view, which is independent of the secrets (Fact 2). The openings $\rho = x - a$ and $\sigma = y - b$ are masked by the uniform a and b. The C opened triples are random and then discarded. And t is 0 in any run that does not abort.

3. Extension 2: the verified shuffle in $\mathbb{Z}_{2^{128}}$

3.1 What the protocol does (crates/mpc/src/mac.rs)

To shuffle columns of values x in $\mathbb{Z}_{2^{64}}$ by a secret π:

1. **Lift.** Read every share as an element of $\mathbb{Z}_{2^{128}}$. This gives sharings of some $x' \equiv x \pmod{2^{64}}$.
2. **MAC.** Draw a random sharing of a key $\alpha \in \mathbb{Z}_{2^{128}}$. Compute $m = \alpha x'$ for every value, by multiplication.

3. **Shuffle.** Run the three reshare steps on values and MACs together.
4. **Coefficients.** Open a fresh random 128-bit seed (checked) and expand it into public coefficients $\beta_k \in \mathbb{Z}_{2^{128}}$.
5. **Check.** Compute $u = \sum_k \beta_k y_k$ and $v = \sum_k \beta_k m_k$ locally, compute αu by multiplication, open $w = \alpha u - v$ (checked), and pass if $w = 0$.
6. **Agree** on the verdict, then reduce every share mod 2^{64}.

3.2 Every deviation is an additive error

A corrupt server can deviate in three places, and each becomes an additive error:

- **The MAC multiplications.** By Fact 2 in $\mathbb{Z}_{2^{128}}$, $m = \alpha x' + g$.
- **The reshare steps.** In the step that excludes P_c, the corrupt server either receives (and its copies do not count) or sends masked summands to the excluded server. Any value it sends is the correct one plus a difference. Later steps only move and remask that difference, so after all three steps:

  $$y_k = x'_{\pi(k)} + e_k, \qquad m_k = \alpha x'_{\pi(k)} + f_k.$$

  Here f includes the permuted MAC-multiplication errors g.
- **The final multiplication.** αu comes out as $\alpha u + d$.

The vectors e and f are fixed before the seed in Step 4 is opened. The value d may depend on the β's, since the corrupt server knows them by then, but not on α.

Which errors matter. After reduction mod 2^{64}, the output is $x_{\pi(k)} + e_k$ mod 2^{64}. So the result is wrong exactly when

$$\exists k : \; e_k \not\equiv 0 \pmod{2^{64}}, \quad \text{i.e.} \quad v_0 := \min_k v_2(e_k) \le 63,$$

where v_2 is the 2-adic valuation, the number of times 2 divides a number, with $v_2(0) = 128$ in this ring. Call such an e *harmful*.

3.3 The opened value

Lemma 3. If no honest server aborts, the opened w is

$$w = \alpha E + D, \qquad E = \sum_k \beta_k e_k, \quad D = d - \sum_k \beta_k f_k.$$

Proof.

By Fact 1, the opening shows the true value of $\alpha u + d - v$.

Expand u with the errors:

$$u = \sum_k \beta_k x'_{\pi(k)} + \sum_k \beta_k e_k.$$

Expand v the same way:

$$v = \alpha \sum_k \beta_k x'_{\pi(k)} + \sum_k \beta_k f_k.$$

Multiply u by α and subtract v. The term $\alpha \sum_k \beta_k x'_{\pi(k)}$ cancels:

$$\alpha u + d - v = \alpha \sum_k \beta_k e_k + d - \sum_k \beta_k f_k = \alpha E + D. \qquad \square$$

3.4 Two counting lemmas

Everything is in $\mathbb{Z}_{2^{128}}$.

Lemma 4 (random coefficients rarely hide an error). Fix e with minimum valuation v_0, and let the β_k be independent and uniform. Then for $0 \le t \le 128 - v_0$:

$$\Pr\left[v_2(E) \ge v_0 + t\right] \le 2^{-t}.$$

Proof.

Pick k^* with $v_2(e_{k^*}) = v_0$, and write

$$e_{k^*} = 2^{v_0} o, \quad o \text{ odd}.$$

Every e_k is divisible by 2^{v_0}, so the rest of the sum is too:

$$\sum_{k \ne k^*} \beta_k e_k = 2^{v_0} R \quad \text{for some } R.$$

Hence

$$E = 2^{v_0}(\beta_{k^*} o + R).$$

Now E is divisible by 2^{v_0+t} exactly when

$$\beta_{k^*} o + R \equiv 0 \pmod{2^t}.$$

Since o is odd it is invertible mod 2^t, so this says

$$\beta_{k^*} \equiv -R\, o^{-1} \pmod{2^t}.$$

Fix all the other β's, which fixes R. Then β_{k^*}, uniform mod 2^{128}, is uniform mod 2^t and hits this one residue with probability 2^{-t}. Averaging over the other β's keeps the bound. $\square$

Lemma 5 (the key catches a visible error). Fix E with $v_2(E) = j < 128$, and any target D. If α is uniform in $\mathbb{Z}_{2^{128}}$ and independent of E and D, then

$$\Pr\left[\alpha E + D = 0\right] \le 2^{j-128}.$$

Proof.

Write

$$E = 2^j o, \quad o \text{ odd}.$$

The equation is

$$2^j \alpha o \equiv -D \pmod{2^{128}}.$$

If 2^j does not divide D, there is no solution, and the probability is 0.

Otherwise divide through by 2^j:

$$\alpha o \equiv -D/2^j \pmod{2^{128-j}}.$$

Since o is invertible, this fixes α mod 2^{128-j} uniquely. A uniform α satisfies it with probability $2^{-(128-j)}$. □

3.5 The bound

Theorem B. If the shuffle introduces a harmful error, the check passes with probability at most

$$\varepsilon_{\text{shuffle}} \le (129 - v_0)\, 2^{v_0 - 128} \le 66 \cdot 2^{-65} < 2^{-58.9},$$

plus the PRF advantage (twice: once for the coefficients, once for hiding α).

Proof.

By Lemma 3, the check passes exactly when $\alpha E + D = 0$.

Split on the valuation of E. Write $j = v_2(E)$, which is always at least v_0:

$$\Pr[\text{pass}] = \sum_{j=v_0}^{127} \Pr[v_2(E) = j] \cdot \Pr[\alpha E + D = 0 \mid v_2(E) = j] + \Pr[E = 0] \cdot \Pr[D = 0 \mid E = 0].$$

Bound each term with $j < 128$. Lemma 5 applies given E, because α is independent of everything the corrupt server knows (below):

$$\Pr[\alpha E + D = 0 \mid v_2(E) = j] \le 2^{j-128}.$$

Lemma 4 with $t = j - v_0$ gives

$$\Pr[v_2(E) = j] \le \Pr[v_2(E) \ge j] \le 2^{v_0 - j}.$$

So each of these terms is at most

$$2^{v_0 - j} \cdot 2^{j-128} = 2^{v_0 - 128}.$$

The last term. When $E = 0$, the corrupt server controls whether $D = 0$, so give it probability 1. Lemma 4 with $t = 128 - v_0$ gives

$$\Pr[E = 0] = \Pr[v_2(E) \ge 128] \le 2^{v_0 - 128}.$$

Add up. There are $128 - v_0$ terms with $j < 128$, plus one more:

$$\Pr[\text{pass}] \le (128 - v_0 + 1)\, 2^{v_0 - 128} = (129 - v_0)\, 2^{v_0 - 128}.$$

This bound grows with v_0, because moving from v_0 to $v_0 + 1$ multiplies it by $2(128 - v_0)/(129 - v_0) > 1$. A harmful error has $v_0 \leq 63$, so the worst case is $v_0 = 63$:

$$66 \cdot 2^{-65} = 2^{-65+\log_2 66} < 2^{-58.9}. \qquad \square$$

Why α is uniform and independent. Its summand from the key the corrupt server lacks is pseudorandom to it. The messages involving α (the MAC multiplications and αu) are masked by fresh zero-sharings (Fact 2's protocol). And α is never opened. So after replacing that key's stream with true randomness (one PRF hybrid), α is uniform and independent of e, f, the β's and d.

Why the β's are uniform. The seed is opened only after e and f are fixed. Replacing its AES expansion with a random function costs the PRF advantage. The distinguisher evaluates the event "$v_2(E) \geq v_0 + t$" for the fixed e, which it can do with oracle access alone.

For comparison. SPDZ2k's Theorem 1 gives $2^{-s+\log_2(s+1)} = 2^{-57.98}$ for $s = 64$. The direct count here is slightly tighter, because this check has one fixed shape.

3.6 No selective failure

This is the property the shuffle check exists for. Asharov et al.'s attack works because whether the protocol aborts can depend on a secret. Here, whether it aborts is decided by $w = \alpha E + D$, and e, f and d are functions of the corrupt server's view and coins, which are independent of the secrets. α and β are independent of the secrets too. So

$$\Pr[\text{abort}] \text{ is the same for every value of the secret inputs.}$$

Errors that are harmless (divisible by 2^{64}) may still cause an abort, and that abort reveals nothing either. A harmful error that is not caught (probability below $2^{-58.9}$) is the only failure, and it is counted in the security error.

4. What the two theorems give together

For one auction with verified product batches $b = 1, \ldots, T$ and S shuffles, a union bound gives:

$$\Pr\left[\text{a wrong value goes undetected}\right] \leq \sum_{b=1}^{T} \frac{N_b}{\binom{N_b B_b + B_b}{B_b}} + S \cdot 2^{-58.9} + \text{PRF advantages.}$$

The bucket size B_b is chosen so that each term of the first sum is at most 2^{-56}. Stage 2 has $S = 34$ shuffles, contributing less than $2^{-53.8}$. The product checks dominate, and the totals the code reports (at least 43.9 bits in every benchmarked configuration) are conservative by the word-counting remark in Section 2.4.

5. What is not proved here

- **A full simulation proof** of the malicious protocol. It would combine Fact 1, Fact 2, Theorem A, Theorem B and the semi-honest simulator (leakage-audit.md) into one simulator that extracts the corrupt server's errors and decides abort the way the real protocol would. The pieces above are the ones that were not covered by a published proof. Putting them together is standard but has not been written out.
- **Abort is not guaranteed unanimous.** The verdict round sends each server's bit to both others. A corrupt server can tell one honest server "pass" and the other "fail". The first then waits for an opening that never completes, so no honest server outputs anything wrong, but the guarantee is security with selective abort, not unanimous abort.
- **Validation openings** reveal a function of a cheating trader's own invalid input, as stated in the threat model.

Sources

- Furukawa, Lindell, Nof, Weinstein. High-Throughput Secure Three-Party Computation for Malicious Adversaries and an Honest Majority. Eurocrypt 2017. https://eprint.iacr.org/2016/944 (Lemma 2.4, Protocol 2.24, Protocol 3.2, Theorem 5.3)
- Asharov, Hamada, Ikarashi, Kikuchi, Nof, Pinkas, Tsuji. Efficient Secure Three-Party Sorting with Applications to Data Analysis and Heavy Hitters. CCS 2022. https://eprint.iacr.org/2022/1595 (Section 5, Protocol 5.4)
- Cramer, Damgård, Escudero, Scholl, Xing. SPDZ2k: Efficient MPC mod 2^k for Dishonest Majority. CRYPTO 2018. https://eprint.iacr.org/2018/482 (Section 3, Claim 1, Theorem 1)
- Eerikson, Keller, Orlandi, Pullonen, Puura, Simkin. Use your Brain! Arithmetic 3PC For Any Modulus with Active Security. ITC 2020. https://eprint.iacr.org/2019/164 (Section 2.4; their triple preprocessing works in a large prime field, not by bucketing in the ring)